PyTorch Image Classification Interview Template Guide

A practical modification guide based on your transfer_learning_template.py / code_template_solid.py

Core idea

If the interviewer gives you a Data folder and asks you to solve an image classification task such as cat/dog, defect/normal, or vehicle/airplane classification, your current template is suitable. The key parts you need to modify flexibly are the dataset path, training configuration, feature-extraction setting, single-image inference path, and dataloader logic for non-standard dataset formats.

- Most common case: the dataset already follows the ImageFolder structure, so you mainly change DATA_DIR.
- Limited time or CPU: set FEATURE_EXTRACT=True and train only the final classifier.
- GPU and larger dataset: set FEATURE_EXTRACT=False and fine-tune the full model.
- Core model change: replace the final ResNet18 fc layer so that its output dimension equals the number of target classes.

1. Confirm this is a classification task

If each image has one image-level label, such as cat, dog, car, or plane, then it is an image classification task. You do not need bounding boxes. You only need image-label pairs. Your template uses datasets.ImageFolder, so it works best with this folder structure:

```
Data/
├── train/
│   ├── cat/
│   │   ├── cat_001.jpg
│   │   └── cat_002.jpg
│   └── dog/
│       ├── dog_001.jpg
│       └── dog_002.jpg
└── val/
    ├── cat/
    │   └── cat_101.jpg
    └── dog/
        └── dog_101.jpg
```

ImageFolder automatically infers class names from subfolder names. For example, train/cat and train/dog will produce class_names = ["cat", "dog"]. Therefore, you can use len(class_names) as the number of output classes.

2. Must-change item 1: DATA_DIR

Original template:

```
DATA_DIR = "hymenoptera_data"
```

If the interview dataset folder is called Data, change it to:

```
DATA_DIR = "Data"
```

Then inspect the dataset structure:

```
print(os.listdir(DATA_DIR))
print(os.listdir(os.path.join(DATA_DIR, "train")))
print(os.listdir(os.path.join(DATA_DIR, "val")))
```

You should confirm that train and val exist, and that the class folders under train and val are consistent.

3. Must-change item 2: training configuration

The template contains these main configuration values:

```
NUM_EPOCHS = 25
BATCH_SIZE = 8
NUM_WORKERS = 0
LEARNING_RATE = 0.001
MOMENTUM = 0.9
STEP_SIZE = 7
GAMMA = 0.1
FEATURE_EXTRACT = False
```

Recommended interview configuration:

```
NUM_EPOCHS = 3
BATCH_SIZE = 8
NUM_WORKERS = 0
LEARNING_RATE = 0.001
MOMENTUM = 0.9
STEP_SIZE = 2
GAMMA = 0.1
FEATURE_EXTRACT = True
```

This is faster, safer on CPU, less likely to overfit, and enough to demonstrate the complete pipeline. If GPU is available and the dataset is reasonably large, you can use FEATURE_EXTRACT=False, NUM_EPOCHS=5, and BATCH_SIZE=16.

4. Must-understand item 3: FEATURE_EXTRACT

In your template:

```
FEATURE_EXTRACT = False
# False: fine-tune the whole ResNet18
# True: freeze backbone and train only the final classifier
```

FEATURE_EXTRACT=True freezes the pretrained ResNet18 backbone and trains only the newly replaced fc layer. This is suitable for small datasets, CPU training, and time-limited interviews.

FEATURE_EXTRACT=False fine-tunes the whole ResNet18. This is suitable when a GPU is available and the dataset is large enough.

Interview explanation:

Since the interview time is limited and the dataset may be small, I freeze the pretrained backbone and only train the final classifier. This makes training faster and reduces overfitting.

5. Core model change: replace the final ResNet18 layer

The key part is build_model():

```
def build_model(num_classes, feature_extract=False):
    model = models.resnet18(weights="IMAGENET1K_V1")

    set_parameter_requires_grad(model, feature_extract)

    num_ftrs = model.fc.in_features
    model.fc = nn.Linear(num_ftrs, num_classes)

    return model
```

The original ResNet18 is trained for 1000 ImageNet classes. For your new task, you keep the pretrained feature extractor and replace the final fully connected layer with a new layer matching the number of target classes.

```
num_classes = len(class_names)
model.fc = nn.Linear(num_ftrs, num_classes)
```

6. Data loading: ImageFolder automatically infers classes

The build_dataloaders() function reads train and val folders and obtains class names automatically:

```
image_datasets = {
```

```
    phase: datasets.ImageFolder(
        root=os.path.join(data_dir, phase),
        transform=data_transforms[phase]
    )
    for phase in ["train", "val"]
}

class_names = image_datasets["train"].classes
```

As long as the folder names are correct, the model can infer the class names and number of classes automatically. You do not need to manually define NUM_CLASSES.

7. Modify the single-image inference path

The original example path is for the ants/bees dataset:

```
test_img_path = os.path.join(
    DATA_DIR,
    "val",
    "bees",
    "72100438_73de9f17af.jpg"
)
```

For a cat/dog task, change it to a real image path:

```
test_img_path = os.path.join(
    DATA_DIR,
    "val",
    "cat",
    "cat_001.jpg"
)
```

If you do not know the image filename, print a few examples:

```
print(os.listdir(os.path.join(DATA_DIR, "val")))
print(os.listdir(os.path.join(DATA_DIR, "val", class_names[0]))[:5])
```

8. What if there is no val folder?

Sometimes the dataset is provided as:

```
Data/
├── cat/
└── dog/
```

In this case, the current build_dataloaders() will not work directly because it expects Data/train and Data/val. You can create a train/validation split using random_split:

```
from torch.utils.data import random_split

def build_dataloaders_auto_split(data_dir, batch_size, num_workers, train_ratio=0.8):
    data_transforms = get_data_transforms()

    full_dataset = datasets.ImageFolder(
        root=data_dir,
        transform=data_transforms["train"]
    )

    class_names = full_dataset.classes
    train_size = int(train_ratio * len(full_dataset))
    val_size = len(full_dataset) - train_size

    train_dataset, val_dataset = random_split(full_dataset, [train_size, val_size])
    val_dataset.dataset.transform = data_transforms["val"]

    dataloaders = {
        "train": torch.utils.data.DataLoader(train_dataset, batch_size=batch_size,
    shuffle=True, num_workers=num_workers),
        "val": torch.utils.data.DataLoader(val_dataset, batch_size=batch_size,
    shuffle=False, num_workers=num_workers),
```

```
    }
    dataset_sizes = {"train": len(train_dataset), "val": len(val_dataset)}
    return dataloaders, dataset_sizes, class_names, data_transforms
```

Do not use this auto-split version if the dataset is already split into train and val.

9. What if labels are stored in a CSV file?

If the dataset is images + labels.csv instead of ImageFolder format, you need a custom Dataset:

```
import pandas as pd
from torch.utils.data import Dataset

class CSVDataset(Dataset):
    def __init__(self, csv_path, img_dir, transform=None):
        self.df = pd.read_csv(csv_path)
        self.img_dir = img_dir
        self.transform = transform
        self.class_names = sorted(self.df["label"].unique())
        self.class_to_idx = {c: i for i, c in enumerate(self.class_names)}

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        img_path = os.path.join(self.img_dir, row["filename"])
        img = Image.open(img_path).convert("RGB")
        label = self.class_to_idx[row["label"]]
        if self.transform is not None:
            img = self.transform(img)
        return img, label
```

However, most interview classification datasets are likely to use ImageFolder because it is the fastest format to implement in PyTorch.

10. Parts you usually do not need to modify

- get_device(): automatically selects CUDA or CPU.
- train_model(): already includes train/eval mode switching, forward pass, loss, backward pass, optimizer.step, validation, and best-checkpoint saving.
- plot_history(): plots loss and accuracy curves.
- visualize_model(): visualizes validation images with predicted and ground-truth labels.
- predict_single_image(): only the image path usually needs to be changed.

11. Interview plan you can say directly

I will solve this as an image classification problem using transfer learning. First, I will inspect the dataset structure and check whether it follows the ImageFolder format with train and validation folders. Then I will define ImageNet-style transforms, load data using ImageFolder, and infer class names automatically. Next, I will use a pretrained ResNet18 model and replace its final fully connected layer according to the number of target classes. If the dataset is small or only CPU is available, I will freeze the backbone and train only the final classifier. Otherwise, I can fine-tune the whole model. I will train using cross-entropy loss, SGD optimizer, and a learning-rate scheduler. Finally, I will evaluate validation accuracy, plot training curves, save the best checkpoint, and run inference on validation images.

12. Common interview questions and answers

Q: Why use pretrained ResNet18?
A: Because it already contains strong visual features learned from ImageNet. Transfer learning is faster and more stable than

training from scratch on a small dataset.

Q: Why replace the final fc layer?
A: The original ResNet18 outputs 1000 ImageNet classes. Our task has a different number of classes, so the final layer must be replaced.

Q: Why use CrossEntropyLoss?
A: It is the standard loss for single-label multi-class classification and combines softmax with negative log likelihood.

Q: Why use different transforms for train and val?
A: Training uses random augmentation to improve generalization. Validation uses deterministic preprocessing for stable evaluation.

13. Final modification checklist

Item	Recommended value / action	Reason
DATA_DIR	"Data"	Point to the interview dataset
NUM_EPOCHS	3	Run the pipeline quickly
BATCH_SIZE	4 or 8	Safer on CPU/Jupyter
NUM_WORKERS	0	Avoid Windows/Jupyter multiprocessing issues
FEATURE_EXTRACT	True	Freeze backbone and train faster
test_img_path	A real validation image path	Show single-image inference

Final recommendation: in the interview, start with FEATURE_EXTRACT=True, NUM_EPOCHS=3, and NUM_WORKERS=0 to make sure the full pipeline runs successfully. If time and GPU resources allow, then try full fine-tuning.